



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

Ching-I Huang

Deep Neuron Network

- Goal: To approximate a mapping function f^* . A feedforward network defines a mapping $y = f(x; \theta)$ and learns the value of the parameters θ that result in the best function approximation.
- The basic neural network model, which can be described a series of functional transformations. Or instance, three functions $f(1), f(2)$, and $f(3)$ can form a chain as $f(x) = f(3)(f(2)(f(1)(x)))$.

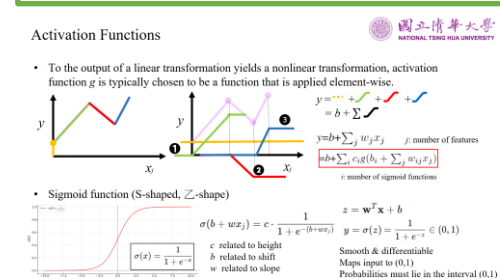
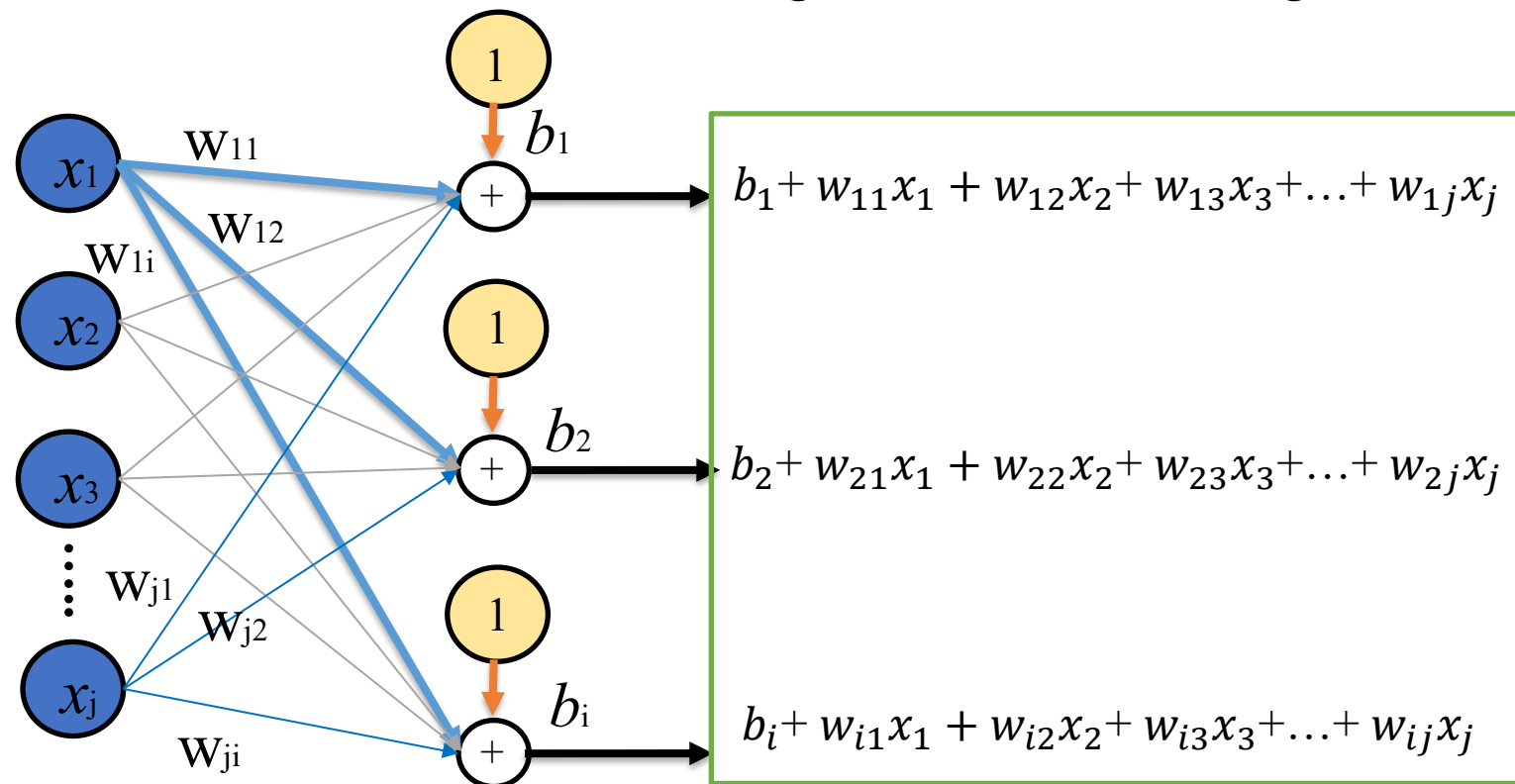
Suppose that we choose a linear model, with θ consisting of bias b and weight w .

$$y = b + \sum_j w_j x_j \quad j: \text{number of features}$$

$$= b + \sum_i c_i g(b_i + \sum_j w_{ij} x_j)$$

i : number of sigmoid functions

$$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_j \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1j} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2j} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3j} \\ w_{i1} & w_{i2} & w_{i3} & \dots & w_{ij} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_j \end{bmatrix}$$



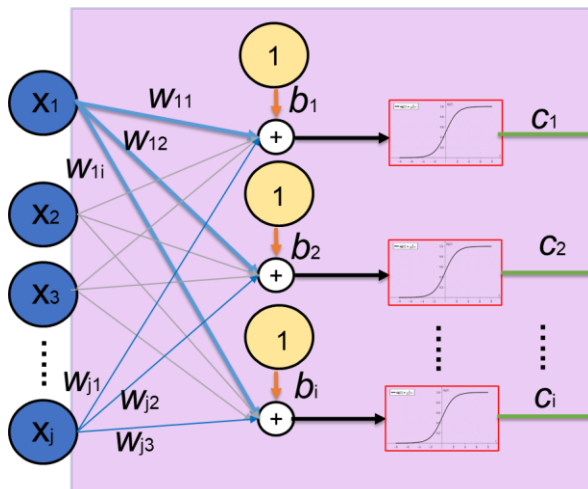
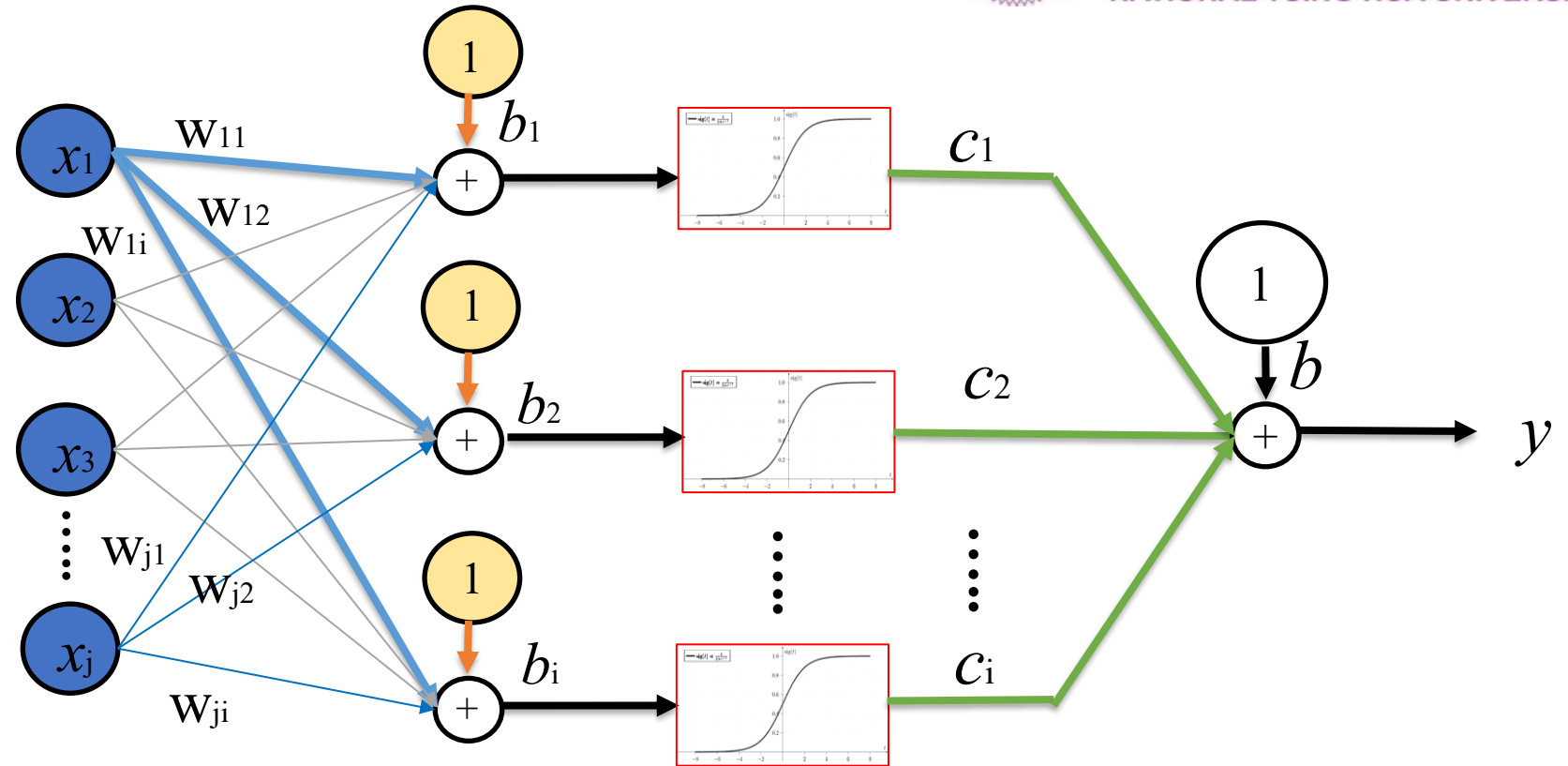


Deep Neuron Network

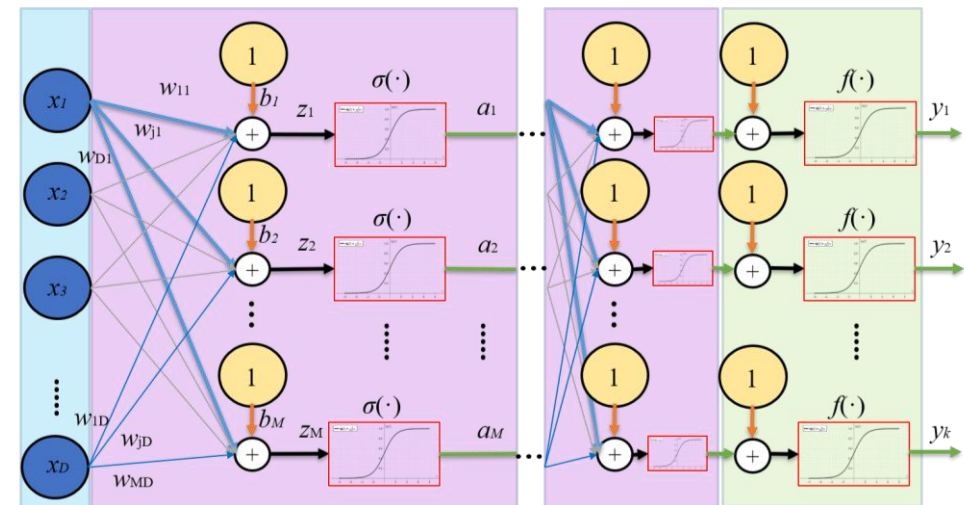
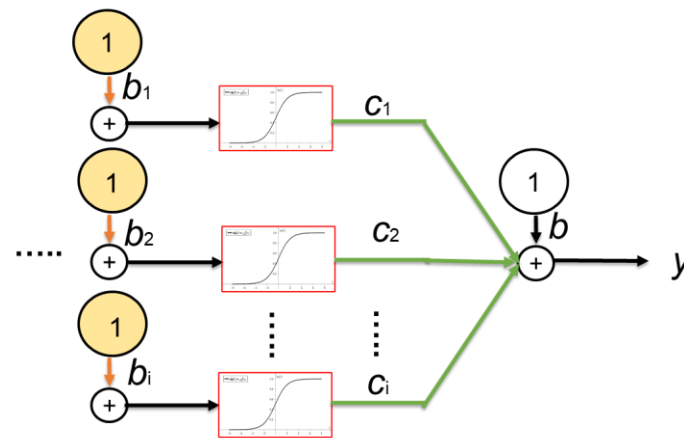
$$y = b + \sum_i c_i g\left(b_i + \sum_j w_{ij} x_j\right)$$

$$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_j \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1j} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2j} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3j} \\ w_{i1} & w_{i2} & w_{i3} & \dots & w_{ij} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_j \end{bmatrix}$$

$$y = b + C^T g(B_{i \times 1} + W_{i \times j} x_{1 \times j})$$

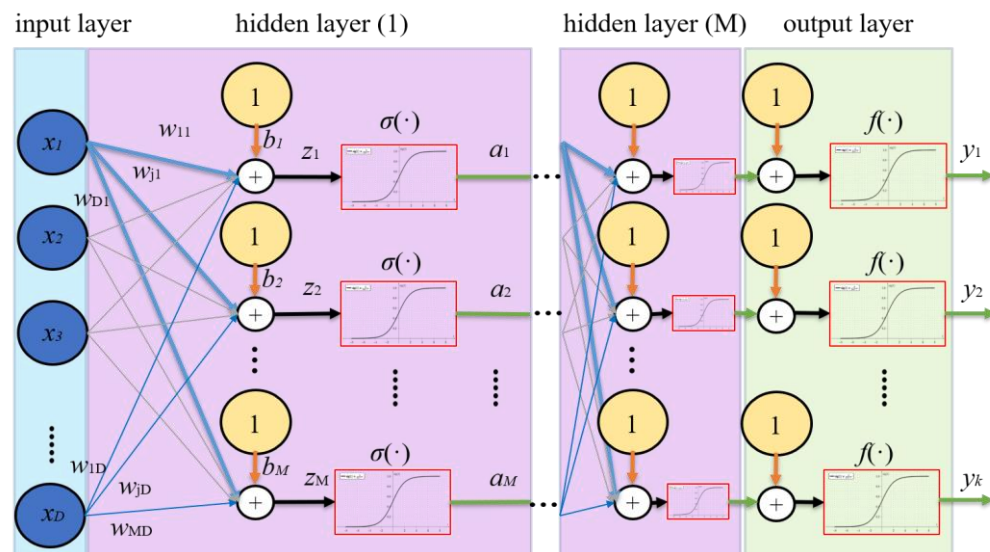


Perceptron, Neuron





Deep Neuron Network



A. Hidden layer

First we construct M linear combinations of the input variables $x_1, \dots, x_i, \dots, x_D$ in the form

$$z_j = \sum_{i=1}^D w_{ji}^{(h)} x_i$$

where the neural unit $j = 1, \dots, M$, and the superscript (h) indicates that the corresponding parameters are in the h^{th} 'layer' of the network. The quantities a_j are known as *activations*. Each of them is then transformed using a

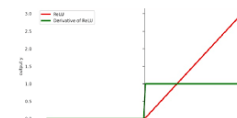
Hidden unit:

激勵函數

Nonlinear functions like the logistic sigmoid or the hyperbolic tangent (tanh) are commonly used as activation functions. Sigmoidal units tend to saturate when z is either very positive or negative, which can hinder gradient-based learning. The hyperbolic tangent, resembling the identity function more closely with $\tanh(0) = 0$ (compared to $\sigma(0) = 1/2$, generally performs better than the logistic sigmoid. However, the widespread saturation (small ∇E) would slow down learning speed. Therefore, many modern neural networks use rectified linear units (ReLU) to avoid these issues while retaining essential properties.

• ReLU:

$$\sigma(\cdot) = \max\{0, z\}$$



ReLU is not differentiable at $z = 0$; software implementations use a one-sided derivative. ReLU outputs zero for half of its domain, allowing its derivatives to remain large and consistent where active. Since the second derivative is always zero, the gradient direction remains more useful for learning compared to activation functions that introduce second-order effects.

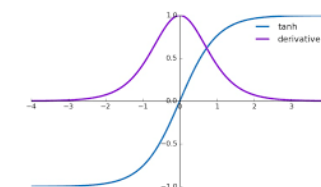
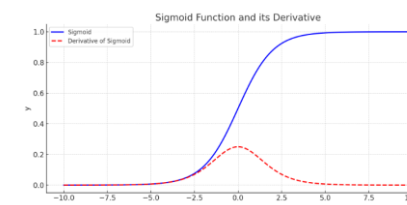
- **Maxout units:** Maxout units generalize rectified linear units (ReLU) further. Each maxout unit divides its input z into groups of k values, and then outputs the maximum element from one of these groups.

$$\sigma(\mathbf{z})_i = \max_{j \in \mathbb{G}^i} z_j$$

where $\mathbb{G}^i$ is the indices of the inputs for group i , $\{(i-1)k+1, \dots, ik\}$. Maxout units can thus be seen as **learning the activation function itself** rather than just the relationship between units. Different examples may update different parts of the network, then, offering some "redundancy" that helps to resist the catastrophic forgetting phenomenon. Each maxout unit is now parametrized by multiple weight vectors instead of one, and implies it requires more training data.

B. Output layer

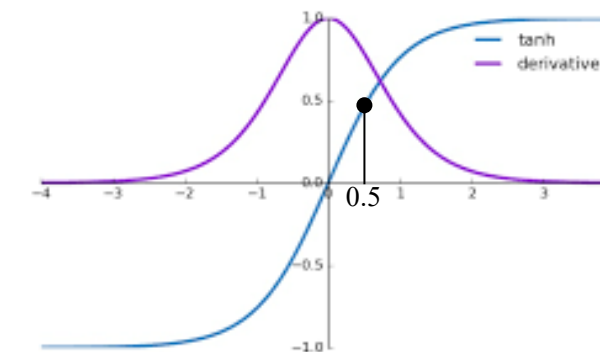
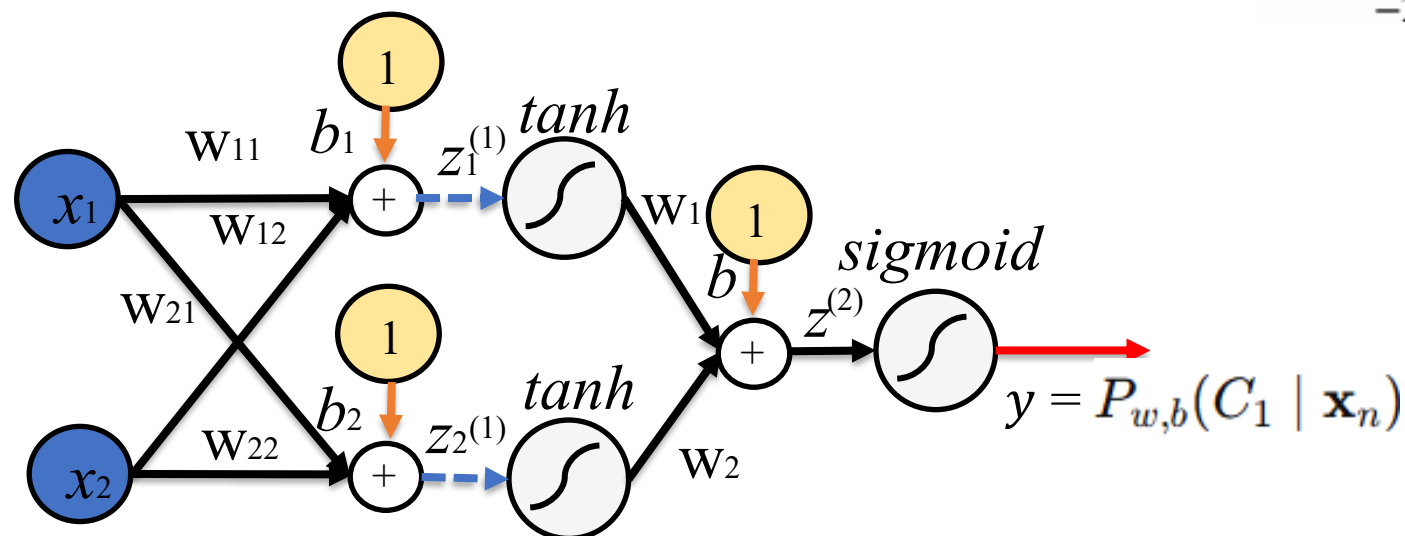
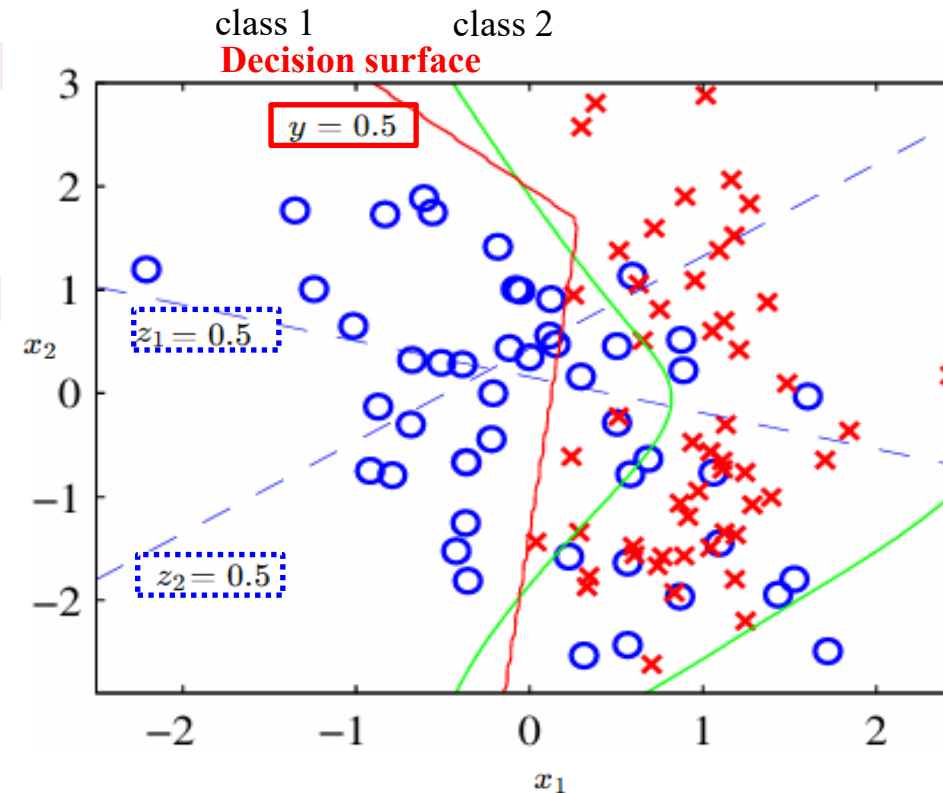
The final layer of a feedforward network is called the output layer. The output activations $f(\cdot)$ are transformed by an activation function to produce network outputs. The choice of activation function depends on the data type and the assumed target distribution.





Deep Neuron Network

Example of the solution of a simple two-class classification problem involving synthetic data using a neural network having two inputs, two hidden units with 'tanh' activation functions, and a single output having a logistic sigmoid activation function. The dashed blue lines show the $z = 0.5$ contours for each of the hidden units, and the red line shows the $y = 0.5$ decision surface for the network. For comparison, the green line denotes the optimal decision boundary computed from the distributions used to generate the data.



activation boundary in the positive region

Network Training (5.2.1)

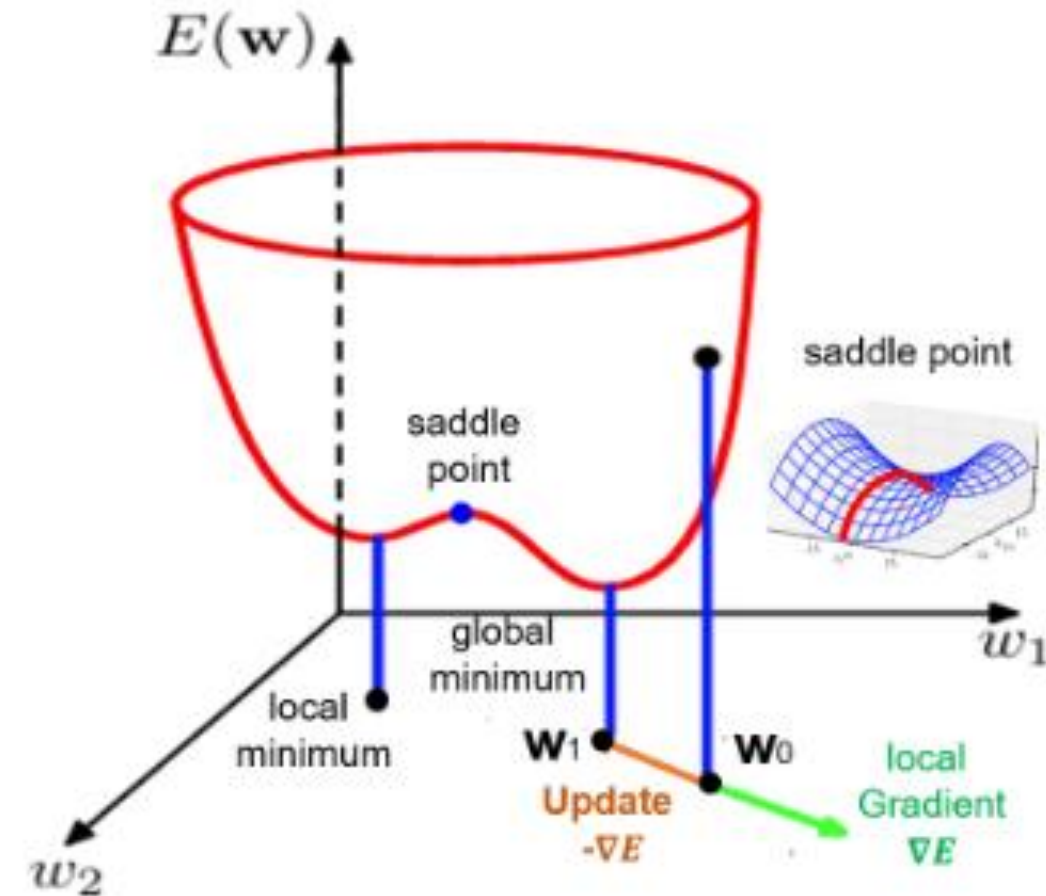
- Goal: to find a weight vector $\mathbf{w}$ such that $E(\mathbf{w})$ takes its smallest value.

A small step in weight space from $\mathbf{w}$, to $\mathbf{w} + \delta\mathbf{w}$ results in a change in the error function $\delta E = \delta\mathbf{w}^T \nabla E(\mathbf{w})$, where $\nabla E(\mathbf{w})$ points in the direction of greatest error increase. The minimum error occurs when the gradient vanishes.

$$\nabla E(\mathbf{w}) = \left[\frac{\partial E(w_1)}{\partial w_1}; \dots; \frac{\partial E(w_M)}{\partial w_M} \right]^T = 0$$

where M indicates that the corresponding parameters.

- Challenge:
 - (1) The error function typically has a highly nonlinear dependence on the weights and bias parameters, and so (2) there will be many points in weight space at which the gradient vanishes (stationary points, such as at minima, maxima, and saddle points) or is numerically very small. Because there is clearly no hope of finding an analytical solution, we resort to iterative numerical procedures.





Network Training (5.2.1)

- Gradient descent optimization:

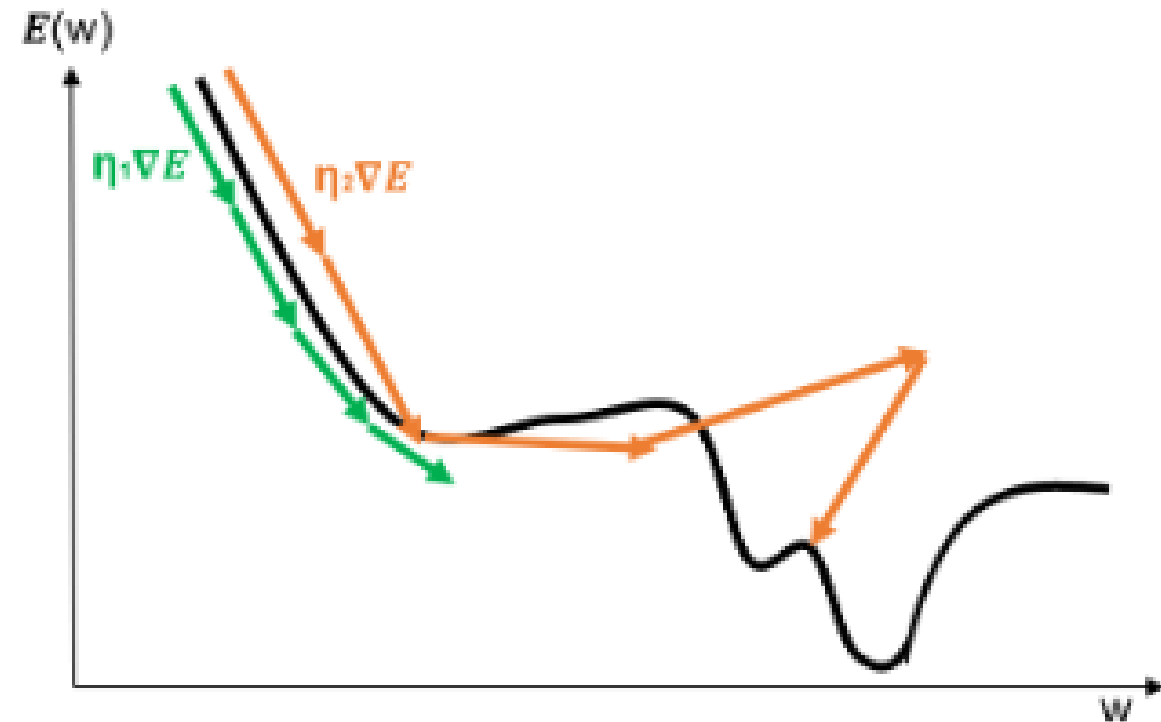
To update the weights, we take a small step in the direction of the negative gradient. To update the weights, we take a small step in the direction of the negative gradient. In practice, even though some units are not differentiable at a few points, gradient descent still performs effectively. Training algorithms typically do not reach a local minimum of the cost function but significantly reduce its value. (sec. 3.1.3 and 5.4.2)

Stochastic gradient descent, SGD, updates the weight vector based on one data point at a time, so that

$$w^{(\tau+1)} \leftarrow w^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

where τ denotes the iteration number, $\eta > 0$ is learning rate parameter and $E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$.

The main challenge is not at critical points, as with many weight parameters, it is difficult to have a local minimum for all of them. Additionally, the optimization process often halts earlier when the decrease in loss becomes insignificant. Instead, plateaus, where the gradient becomes very small and progress stalls, are a more common issue.



Hyperparameters refer to the settings that govern the behavior of the learning algorithm; however, the values of hyperparameters are not adjusted by the learning algorithm itself.

Mini-batch Gradient Descent

A compromise method: each update uses a small batch of data (instead of the entire dataset or just one sample).

Batch sizes are often 32, 64, 128.... (powers of 2)



Network Training (5.2.1)

Optimization:

A popular algorithm Adam (adaptive momentums) is combination of RMSProp (Root Mean Square Propagation) and Momentum. Momentum approach avoid getting stuck in critical points.

- **Momentum:** This method continues the movement from the previous iteration, adjusted by the negative gradient:

$$\begin{aligned} v^{(\tau+1)} &\leftarrow \lambda v^{(\tau)} - (1 - \lambda) \nabla E_n^{(\tau)} \\ w^{(\tau+1)} &\leftarrow w^{(\tau)} + \eta v^{(\tau+1)} \end{aligned}$$

decay factor

where λ is a hyperparameter, and $v^{(\tau+1)}$ is a moving average of $-\nabla E_n^{(\tau)}$. Momentum v is the accumulation of all previous gradients, with earlier gradients weighted less than more recent ones. Momentum helps accelerate convergence by maintaining the direction of the previous gradient. while the Lookahead method updates the gradient descent: $v^{(\tau+1)} = \lambda v^{(\tau)} - (1 - \lambda) \nabla E_n^{(\tau+1)}$ to accelerate convergence toward a minimum.

Using a smaller learning rate in steep directions prevents overshooting, while a larger learning rate in flat directions speeds up convergence.

Adaptive Gradient Algorithm

- **Adagrad:** The learning rate for each parameter is divided by the root mean square of its previous derivatives:

$$\begin{aligned} \eta^{(\tau)} &\leftarrow \frac{\eta}{\sqrt{\tau + 1}} \\ \mathbf{r}^{(\tau+1)} &\leftarrow \mathbf{r}^{(\tau)} + \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)} \end{aligned}$$

time decay

Hadamard product (element-wise product)

$$(A \odot B)_{ij} = (A)_{ij}(B)_{ij}.$$

the root mean square of accumulated gradients

$$r^{(\tau+1)} = \sum_{i=0}^{\tau} (\nabla E_n^{(i)})^2$$

The update term can be expressed as:

$$\begin{aligned} \frac{\eta^{(\tau)}}{\sqrt{r^{(\tau+1)}}} &= \frac{\eta}{\sqrt{\tau + 1}} \odot \frac{1}{\sqrt{\frac{1}{\tau+1} \sum_{i=0}^{\tau} (\nabla E_n^{(i)})^2}} \\ &= \frac{\eta}{\sqrt{\sum_{i=0}^{\tau} \nabla E_n^{(i)} \odot \nabla E_n^{(i)}}} \end{aligned}$$

The first term reduces the learning rate in all directions as iterations increase (time decay), while the second term adjusts the learning rate more aggressively in directions with gentler slopes. Despite the time decay in the first term, the overall effect emphasizes the relative change compared to the total previous updates along the same direction. The update depends on the direction of the parameter change, which can lead to premature adaptivity, as the optimal learning rate in a given direction may shift over time.

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \frac{\eta}{\sqrt{r^{(\tau+1)}} + \epsilon} \odot \nabla E_n^{(\tau)}$$

approximating the the ratio between the first derivative and the second derivative.

$$\frac{\eta}{\sqrt{\sum_{i=0}^{\tau} \nabla E_n^{(i)} \odot \nabla E_n^{(i)}}} \odot \nabla E_n^{(\tau)}$$

the square root of the accumulated average of past squared first derivatives.

the current first derivative (the gradient)

- Each parameter has its own learning rate. (element-wise product)
- Automatically adapts learning rates, especially useful for sparse data
- Step size depends on gradient magnitude, not update count.
- emphasizes the **relative change** of the gradient rather than its absolute value, which highlights variations in the gradient over time.



Network Training (5.2.1)

- In **Adagrad**, the denominator grows continuously over time, which makes the learning rate shrink and may lead to premature stopping.

$$\mathbf{r}^{(\tau+1)} \leftarrow \mathbf{r}^{(\tau)} + \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)}$$

$$r^{(\tau+1)} = \sum_{i=0}^{\tau} (\nabla E_n^{(i)})^2$$

- Effective for **sparse features** and **high-dimensional data**.
- Valuable in NLP: Frequent words (the, is, and...) get smaller updates.
Rare words (mitochondria, quasar...) retain larger learning rates.
- Learning rate keeps decreasing over time (monotonic decay).
- RMSProp** replaces r with an **exponential moving average** so that it only considers the recent average of squared gradients.

$$r^{(\tau+1)} = \lambda r^{(\tau)} + (1 - \lambda)(\nabla E_n^{(\tau)})^2$$

Exponential decay

$$r^{(\tau+1)} = (1 - \lambda)(\nabla E_n^{(\tau)})^2 + \lambda(1 - \lambda)(\nabla E_n^{(\tau-1)})^2 + \dots$$

$$\mathbf{w}^{(\tau+1)} = \lambda \mathbf{w}^{(\tau)} - (1 - \lambda) \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} \odot \nabla E_n^{(\tau)}$$

Adam – Adaptive Moment Estimation

Then, The Adam optimizer combines the advantages of both RMSProp and Momentum by maintaining moving averages of the gradients and their second moments, which use exponential decay rates to apply bias corrections.

$$\mathbf{v}^{(\tau+1)} = \lambda_1 \mathbf{v}^{(\tau)} + (1 - \lambda_1) \nabla E_n^{(\tau)}$$

$$\mathbf{r}^{(\tau+1)} = \lambda_2 \mathbf{r}^{(\tau)} + (1 - \lambda_2) \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)}$$

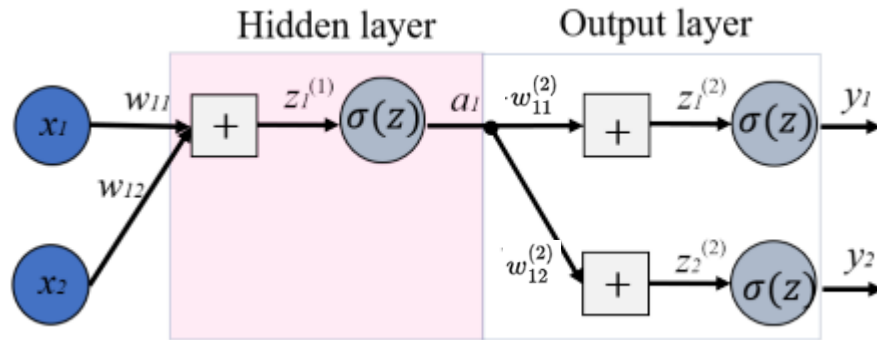
$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} \mathbf{v}^{(\tau+1)}$$

This allows for adaptive learning rates across different parameters, improving convergence. Adam adjusts the learning rate for each parameter based on the estimates of the first and second moments of the gradients, making it more resilient to noisy gradients and suitable for handling sparse data.



Network Training (5.3.1)

- Error backpropagation



$$\frac{\partial E_n}{\partial w_{11}} = ?$$

Chain rule

$$\frac{\partial E_n}{\partial w_{11}} = \frac{\partial E_n}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}}$$

$$\begin{cases} z_1^{(1)} = w_{11}x_1 + w_{12}x_2 \\ \frac{\partial z_1^{(1)}}{\partial w_{11}} = x_1 \end{cases}$$

Chain rule

$$\frac{\partial E_n}{\partial z_1^{(1)}} = \frac{\partial E_n}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} = \frac{\partial E_n}{\partial a_1^{(1)}} \sigma'(z_1^{(1)})$$

Chain rule

$$\frac{\partial E_n}{\partial a_1^{(1)}} = \frac{\partial E_n}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \frac{\partial E_n}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} = \frac{\partial E_n}{\partial z_1^{(2)}} w_{11}^{(2)} + \frac{\partial E_n}{\partial z_2^{(2)}} w_{12}^{(2)}$$

Next layer is output layer

$$\begin{cases} \frac{\partial E_n}{\partial z_1^{(2)}} = \frac{\partial E_n}{\partial y_1} \frac{\partial y_1}{\partial z_1^{(2)}} \\ \frac{\partial E_n}{\partial z_2^{(2)}} = \frac{\partial E_n}{\partial y_2} \frac{\partial y_2}{\partial z_2^{(2)}} \end{cases}$$

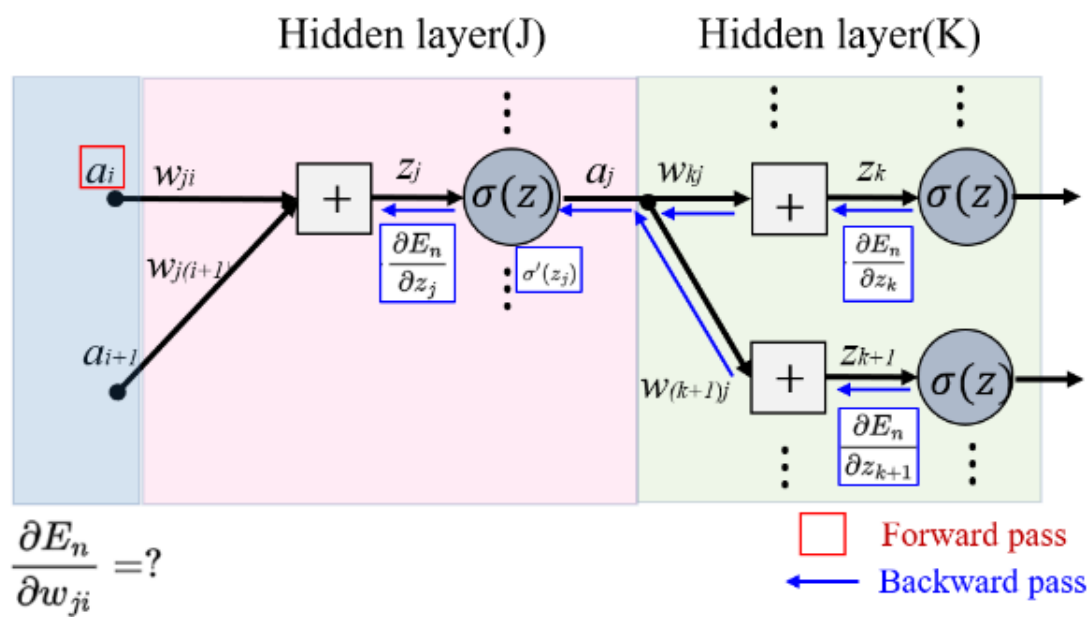
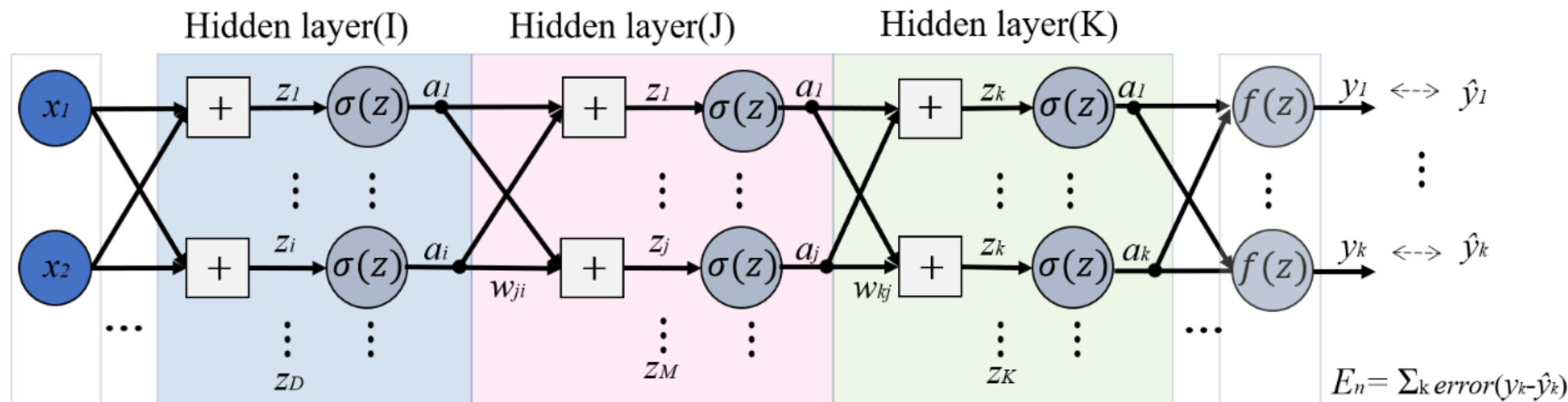
or hidden layer

$$\begin{cases} \frac{\partial E_n}{\partial z_1^{(2)}} = \frac{\partial E_n}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ \frac{\partial E_n}{\partial z_2^{(2)}} = \frac{\partial E_n}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{cases}$$



Network Training (5.3.1)

- Error backpropagation



Chain rule

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial z_j} \frac{\partial z_j}{\partial w_{ji}}$$

$$\begin{cases} z_j = \dots + w_{ji}a_i + w_{j(i+1)}a_{i+1} + \dots \\ \frac{\partial z_j}{\partial w_{ji}} = a_i \end{cases}$$

Chain rule

$$\frac{\partial E_n}{\partial z_j} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial z_j} = \frac{\partial E_n}{\partial a_j} \sigma'(z_j)$$

Chain rule

$$\begin{aligned} \frac{\partial E_n}{\partial a_j} &= \dots + \frac{\partial E_n}{\partial z_k} \frac{\partial z_k}{\partial a_j} + \frac{\partial E_n}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial a_j} + \dots = \sum_k \frac{\partial E_n}{\partial z_k} \frac{\partial z_k}{\partial a_j} \\ &= \sum_k \frac{\partial E_n}{\partial z_k} w_{kj} \end{aligned}$$

Next layer (K) is output layer or hidden layer

$$\frac{\partial E_n}{\partial z_k} = \frac{\partial E_n}{\partial y_k} \frac{\partial y_k}{\partial z_k} \quad \text{or} \quad \frac{\partial E_n}{\partial z_k} = \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial z_k}$$

$k = 1, 2, \dots, K$

$$\rightarrow \frac{\partial E_n}{\partial w_{ji}} = a_i \sigma'(z_j) \sum_k w_{kj} \frac{\partial E_n}{\partial z_k}$$

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}$$